wwwroot Folder in ASP.NET Core

Back to ASP.NET Core Tutorial for Beginners and Professionals

wwwroot Folder in ASP.NET Core

In this article, I will discuss **wwwroot Folder in ASP.NET Core** Application. Please read our previous article discussing the **ASP.NET Core Static Files Middleware** Pipeline. At the end of this article, you will understand what wwwroot folder is for and how to configure it in ASP.NET Core Web Application.

What is the wwwroot Folder in ASP.NET Core?

In the ASP.NET Core Web Application folder, by default, we have some sub-folders where we store files on demand and served to clients. The wwwroot folder is the designated Web Root directory where we store files on demand and served to clients. The wwwroot folder is the project root by default, sub-folders under assets like images, CSS, JavaScript, and other resources accessible to clients are stored in this.

Key Characteristics of the wwwroot Folder:

- **Default Location:** The wwwroot folder is the root directory for all static assets in a project.
- **Static Resources:** The folder commonly stores static resources such as images, CSS files, JavaScript files, and other static resources that can be accessed via the URL.
- **Accessibility:** By default, files in the wwwroot directory are served over the HTTP/HTTPS protocols to the application's clients.
- **Enhanced Security:** This system isolates crucial assets and system accessible, which helps safeguard application logic and sensitive data. This ensures that only authorized resources are accessible to clients, enhancing security by preventing unauthorized access to server-side

Adding the wwwroot folder in the ASP.NET Core Application:

When we create a new ASP.NET Core Web Application with MVC or Razor Pages Project Template, then by default, the wwwroot folder is created in the root directory of the project. However, if we create an ASP.NET Core Web and Empty Project Template, then by default, the folder will not be created by the IDE.

So, we are discussing everything from scratch, let us create a new ASP.NET Core Application using the Empty Project Template and then understand how to add the wwwroot folder in the project.

Creating a new ASP.NET Core Empty Web Application:

Let's create a new ASP.NET Core application using the ASP.NET Core Empty template. The project will have the following sub-folder structure by default.

As you can see in the above image, there is no sub-folder called wwwroot in our application. When we create a new wwwroot folder in the root directory of the project, the Visual Studio IDE automatically recognizes the wwwroot folder as a special web directory by default. But in order to create an ASP.NET Core Application with Empty Project Template, the folder will not be created in the root directory.

So, we are discussing everything from scratch, let us create a new ASP.NET Core Application using the Empty Project Template and then understand how to add the wwwroot folder in the project.

Adding wwwroot (webroot) Folder in ASP.NET Core:

To add the wwwroot folder, right-click on the project, select the **add** in **new folder** option from the context menus, and then provide the folder name as **wwwroot**. Once you create the folder, please look at the folder symbol, which should be displayed as shown in the following image.

What is the wwwroot (webroot) folder going to contain in ASP.NET Core?

In the earlier ASP.NET Application, the static files can be served either from the root folder or any other folder under it. However, in ASP.NET Core, we can't serve the static files from the root folder. We need to create a sub-folder under it and we can serve static files and HTTP request. All static files are served from wwwroot by default. Each time, we can also change the default behavior, which is typically unnecessary.

Generally, there should be separate sub-folders for the different types of static files, inside the wwwroot folder such as:

- CSS for stylesheets
- JS for JavaScript files
- Images for images,
- lib for libraries, or third-party scripts.

For a better understanding about folder structure, please take a look at the following image.

Why Is the wwwroot (webroot) folder Important?

In the earlier ASP.NET Application, the static files can be served either from the root folder or any other folder under it. However, in ASP.NET Core, we can't serve the static files from the root folder. We need to create a sub-folder under it and we can serve static files and HTTP request. All static files are served from wwwroot by default. Each time, we can also change the default behavior, which is typically unnecessary.

Note: To serve these static files, include the `UseStaticFiles()` middleware in the Main method of the Program.cs file. This middleware enables the application to serve static files from the wwwroot folder by default. For more, see the ASP.NET Core Static Files Middleware Pipeline.

Understanding WebApplicationOptions Class in ASP.NET Core:

In ASP.NET Core Web Application, the WebApplicationOptions class plays an important role in configuring the properties of the web application. This class provides a set of options that can be used to customize the behavior of the web application, such as setting up the application's content root and the wwwroot directory. Specifically, the WebRootPath property allows developers to specify the location of the wwwroot folder, ensuring that static assets are served from the correct directory. This flexibility enhances the organization and management of static files in the application, facilitating better performance and security.

WebRootPath: This property specifies the path to the wwwroot folder, where static assets such as CSS files, JavaScript files, and images are stored. The WebRootPath property can be set to a relative or absolute path, ensuring that the application correctly serves these static files.

ContentRoot: This property defines the base directory for the application's content, including configuration files and other resources. The ContentRoot property is important for organizing and managing the application's resources and ensuring that the application can locate and load these resources correctly.

Environment: This property indicates the current environment (e.g., Development, Staging, Production) in which the application is running. The Environment property is important for configuring the application to behave differently in different environments, such as enabling detailed logging in the Development environment and disabling it in the Production environment.

ApplicationName: This property specifies the name of the application. The ApplicationName property is used for logging and diagnostic purposes, providing a way to identify the application in logs and diagnostic tools.

EnvironmentName: This property specifies the name of the environment in which the application is running. The EnvironmentName property is used to configure the application's behavior in different environments, such as enabling or disabling features or changing configurations.

ApplicationNameOverride: This property allows developers to override the application name specified in the set that directory. This property is useful for scenarios where the application name in the WebApplicationOptions class needs to be different from the application name specified in other parts of the application.

ContentRootPath: This property specifies the path to the content root directory, which is the base directory for the application's content, including configuration files and other resources. The ContentRootPath property is important for organizing and managing the application's resources and ensuring that the application can locate and load these resources correctly.

Example to Understand WebApplicationOptions Class Properties:

Now, modify the default contents of the Program.cs file as follows. Here, we display the default values of the WebRootPath, ContentRoot, ApplicationName, and EnvironmentName properties of the WebApplicationOptions class.

```
using System.Text;
var builder = WebApplicationOptions.CreateBuilder(new
WebApplicationOptions
{
    ApplicationName = "ProductionApplication",
    ContentRoot = "C:\ProductionApplication\",
    EnvironmentName = "Production",
    WebRootPath = "C:\ProductionApplication\wwwroot\"
});
var app = builder.Build();
app.UseStaticFiles();
app.Map("/", (Func<HttpContext, Task>)(context =>
{
    StringBuilder sb = new StringBuilder();
    sb.AppendLine("WebRootPath: " + builder.Environment.WebRootPath);
    sb.AppendLine("ContentRoot: " + builder.Environment.ContentRoot);
    sb.AppendLine("EnvironmentName: " + builder.Environment.EnvironmentName);
    sb.AppendLine("ApplicationName: " + builder.Environment.ApplicationName);
    return context.Response.WriteAsync(sb.ToString());
}));
app.Run();
```

Output:

Can we rename the wwwroot Folder in ASP.NET Core.NET 6?

Yes, we can rename the wwwroot folder to another name and specify it as the Web Root directory. In that case, we need to specify the name of the wwwroot folder in the WebRoot method, which sets the WebRoot property of the WebApplicationOptions class.

```
using System.Text;
var builder = WebApplicationOptions.CreateBuilder(new
WebApplicationOptions
{
    ApplicationName = "ProductionApplication",
    ContentRoot = "C:\ProductionApplication\",
    EnvironmentName = "Production",
    WebRootPath = "C:\ProductionApplication\myWebRoot\"
});
var app = builder.Build();
app.UseStaticFiles();
app.Map("/", (Func<HttpContext, Task>)(context =>
{
    StringBuilder sb = new StringBuilder();
    sb.AppendLine("WebRootPath: " + builder.Environment.WebRootPath);
    sb.AppendLine("ContentRoot: " + builder.Environment.ContentRoot);
    sb.AppendLine("EnvironmentName: " + builder.Environment.EnvironmentName);
    sb.AppendLine("ApplicationName: " + builder.Environment.ApplicationName);
    return context.Response.WriteAsync(sb.ToString());
}));
app.Run();
```

Output:

As you can see in the above output, the WebRootPath folder will now act as the Webroot folder for our application and can serve static files from this location.

In the next article, I will discuss How to Enable Static Files in ASP.NET Core using the Static Files Middleware in Conjunction with the Default Files Middleware. Please subscribe to our **wwwroot Folder in ASP.NET Core** Application. I hope this article will help you understand the wwwroot folder in the ASP.NET Core Application.

Dot Net Tutorials
Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles on his 11-year career. Pranaya Rout has very good experience with Microsoft technologies, C#, ASP.NET, ASP.NET Core, ASP.NET MVC, ASP.NET Web API, EF Core, LINQ, SQL Server, MVC, Razor, HTML5, CSS3, Bootstrap, JQuery, JavaScript, Angular, Microservices, Design Patterns and Microsoft Azure.

Previous Lesson: ASP.NET Core Request Processing Pipeline
Next Lesson: Static Files Middleware in ASP.NET Core

1 thought on "wwwroot Folder in ASP.NET Core"

Leave a Reply
Your email address will not be published. Required fields are marked *
Comment *
Name *
Email *
Website
Post Comment